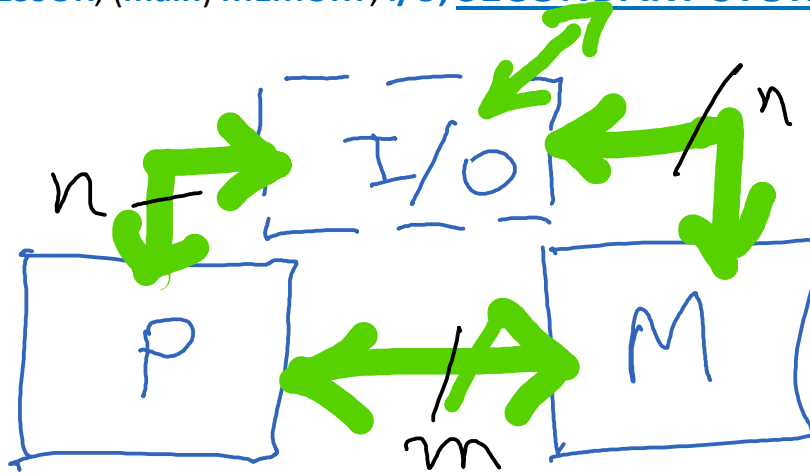


LECTURES 11, 12

What is a **COMPUTER SYSTEM**?

PROCESSOR, (main) **MEMORY**, I/O, **SECONDARY STORAGE**



What is a **bit**? What is a **byte**? What is the meaning of **information**?

What is the role of **clock** in a digital circuit? What is the meaning of **clock speed**?

Within a computer **system**, what is a **bus**? Diagram above shows a system with three different internal busses, and one external bus.

Simple calculation:

What is the maximum data transfer rate over a data bus which is 32 bits wide, and runs at 500 MHz?

Answer: 2 gigabytes/second = 16 gigabits/second.

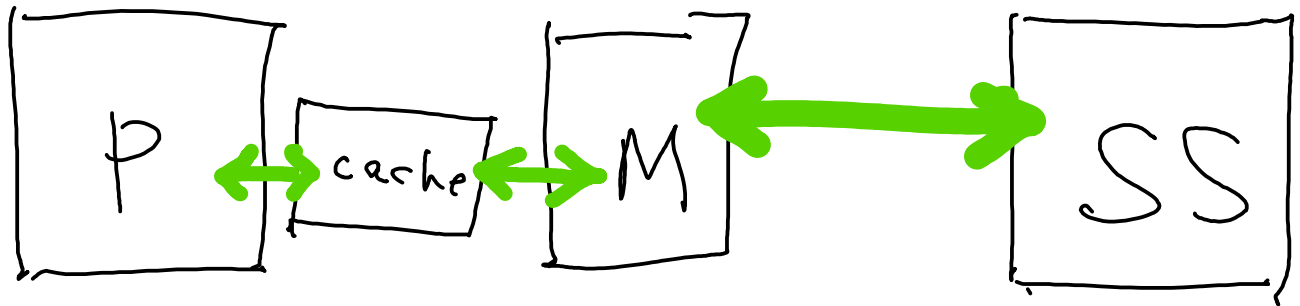
Serial versus **parallel** mode of data transfer. What is the full form of **USB**?

Meaning of data lines, control lines, power, ground.

See notation above indicating **bus width**.

Meaning of **DIRECT MEMORY ACCESS**.

MEMORY HIERARCHY



Processor, main memory and SS are produced using different types of technology. Processor must provide very high processing speed, but memory needs to provide very high storage density with low power consumption.

There is always a speed mismatch between processor, main memory, and SS. For example, the processor may run with a 1 nanosecond clock, whereas main memory may have 30 nanosecond cycle time. SS is generally slower.

Question: What are the typical contents of main memory when the computer is running? Answer: operating system (OS), executable (binary) user programs, and data for the OS and executable programs. The source program and executable program remain in your folder on secondary storage. The executable program is copied into main memory when the user decides to execute – that is, run – the program.

SS reads/writes N **blocks** of data at a time.

Large files and databases remain on secondary storage. Selected blocks of files or databases from secondary storage (SS) are brought into main memory as and when needed, and written back to SS when they are not needed.

Block sizes on SS are usually a few KB. Data can only be read/written from/to SS in an integer number of blocks. Buffers in main memory intervene between executing programs and SS. Recall the working of file systems.

B-TREE – a data structure for efficient large scale storage / retrieval of records. Balanced search tree which takes into account the characteristics of SS devices. Can be viewed as a clever generalization of the binary search tree. Node size of the data structure matches the block size of SS.

A **B-TREE** is a rooted tree T with the following properties:

1. Every node X has the following fields:

- integer $n[X]$, the number of keys stored in X ,
- $n[X]$ key values stored at the node: $key_1[X], key_2[X] \dots key_{n[X]}[X]$ in **non-decreasing** order, and
- flag $leaf[X]$.

2. If X is an internal node, then $n[X]+1$ pointers: $c_1[X], c_2[X] \dots c_{n[X]+1}[X]$ to its child nodes.

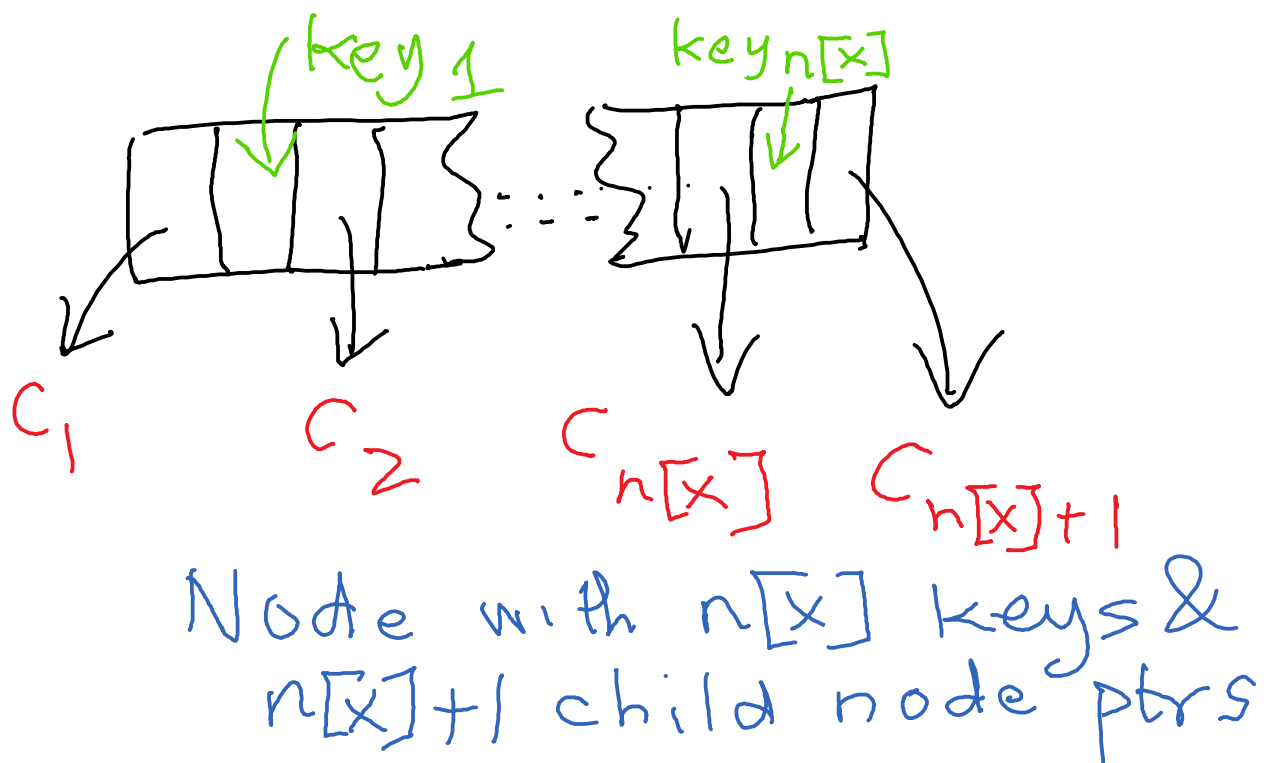
3. The key values $key_1[X], key_2[X] \dots key_{n[X]}[X]$ **separate** the $n[X]+1$ ranges of keys stored in each subtree.

[See figure. Note how pointers and keys alternate.]

4. The B-tree is height-balanced at all times.

5. An integer $t \geq 2$ defines the **minimum degree** of the B-tree.

- Every node other than $root[T]$ has at least $t-1$ keys, and therefore at least t child nodes.
- Every node has at most $2t-1$ keys. A node with $2t-1$ keys is full.



The number of SS accesses required for most operations on a B-tree is proportional to tree height h .

Theorem: For a B-tree with minimum degree $t \geq 2$, and $n \geq 1$ nodes, the tree height h satisfies: $h \leq \log_t[(n+1)/2]$.

B-TREE-SEARCH(x, k)

```
search for key value k among  $key_i[x]$ ,  $i = 1, 2 \dots$ 
  until  $k > key_i[x]$ ;

if  $k = key_i[x]$  // key found at  $x, i$ 
  return( $x, i$ );

if  $x$  is a leaf node // key not found
  return NIL;

SS-READ(  $c_i[x]$  ); // get required child node

return B-TREE-SEARCH(  $c_i[x], k$  ); // recurse
```



Search proceeds from root to leaf. Number of SS blocks (or 'pages') accessed is $\Theta(h)$. Note that this function does not have binary search in the first step.

Creating an empty B-tree: **simple**.

INSERTING A KEY

BASIC IDEA:

CASE 1: In non-full leaf node, insertion is straightforward. Else, we search recursively for the (non-full) leaf node in which to insert key k .

B-TREE-INSERT-NONFULL(s, k)

CASE 2: If a node is FULL, it cannot accommodate one more key. So the node must be **SPLIT** into two adjacent siblings of the parent node. The parent then has one more key, and (recursively) the parent MAY also have to be **SPLIT**.

B-TREE-SPLIT-CHILD(x, i, y) // y is i^{th} child of x

CASE 3: The tree height increases IF the root node needs to be **SPLIT**, in which case a new root node with two child nodes is added.

Overall:

B-TREE-INSERT (T, k)

$r = \text{root}[T];$

if r **is FULL**

create new parent node s **of** r ;

B-TREE-SPLIT-CHILD ($s, 1, r$);

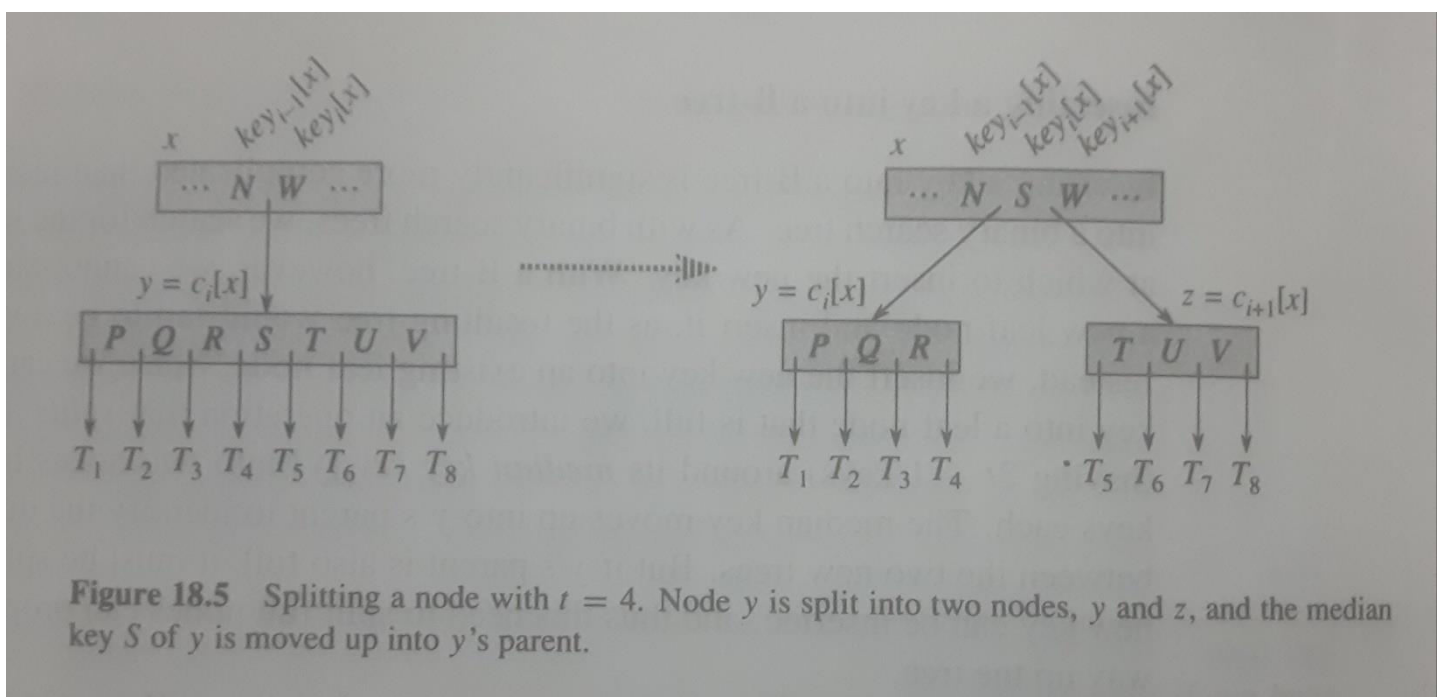
B-TREE-INSERT-NONFULL (s, k);

else

B-TREE-INSERT-NONFULL (r, k);

Number of SS blocks read/written is $\Theta(h)$. An SS block which is not modified need not be written back.

Example of splitting a node:



[Satellite data: Must be handled as we saw in BST.]